武汉大学计算机学院
本科生课程设计报告

HealthMate 需求规格说明

专业: 计算机弘毅班

课程: 计算机综合项目实践

指导教师: 谭小琼

团队名称: token

团队成员: 黎宇恒、周炜

学年学期: 2025-2026 学年第二学期

武汉大学计算机学院
2025 年 3 月 16 日

目录

1 引言 3
1.1 编写目的 3
1.2 产品范围 3
1.3 文档约定 3
2 用户需求说明 4
2.1 用户群体分析 4
2.2 核心业务需求（User Stories） 4
2.3 核心业务流程 5
3 需求分析建模 7
4 功能需求规格 27
4.1 账号与基础档案管理模块 27
4.2 多模态健康数据录入模块 27
4.3 AI 智能分析与建议模块 28
4.4 任务目标与打卡管理模块 28
4.5 健康数据可视化模块 29
4.6 系统后台定时任务模块 29
5 非功能性需求 31
5.1 性能需求 31
5.2 安全需求 31
5.3 可用性需求 31
5.4 可靠性需求 32
5.5 可维护性需求 32
5.6 可扩展性需求 32
6 系统容错与安全设计 34
6.1 数据录入容错：模糊输入解析+用户修正权 34
6.2 断签容错：优雅降级+有效样本趋势分析 34
6.3 AI 服务容错：Mock 兜底+流式输出 34
6.4 长期记忆容错：记忆压缩+Prompt 分层组装 34
6.5 任务管理容错：动态难度调整+目标版本管理 35
6.6 医疗安全容错：词汇拦截+Prompt 强约束 35
7 开发与部署环境 36
7.1 开发环境 36
7.2 部署环境 36
7.3 部署流程 37

1 引言

1.1 编写目的

本文档明确 HealthMate **健康伴侣**的业务需求、功能需求、非功能需求及系统容错设计，作为前后端开发、API 设计、数据库设计、系统测试与部署的核心依据，确保团队对产品目标、技术实现与工程规范的理解高度一致。

本文档整合了前期需求分析中对**模糊输入解析、数据稀疏容错、AI 医学安全、长期记忆管理**等核心痛点的解决方案，形成可落地的最终版需求规范。

1.2 产品范围

HealthMate 是一款聚焦数据记录-智能分析-干预打卡-趋势反馈闭环的个人健康管理应用，以大语言模型（LLM）为核心，承担**模糊特征提取器**与**个性化干预引擎**的角色。

系统支持多模态极简数据录入，结合 RAG 检索增强生成技术与用户健康档案、历史数据，生成具备**连续性、安全性、情绪价值**的个性化健康建议，并将建议转化为结构化打卡任务；同时实现健康数据可视化、长期记忆管理与系统级容错兜底，解决传统健康工具录入门槛高、建议泛化、断签失效等痛点。

产品定位为**非医疗级日常健康管理工具**，面向亚健康人群、健身小白与上班族，不提供疾病诊断与医疗处方服务。

1.3 文档约定

1. 本文档中「必须」表示强约束需求，开发与测试需 100%实现；「建议」表示优化型需求，优先保障核心功能后落地；

2. 涉及的技术栈与第三方服务为开发基准，可在保证功能一致的前提下替换同等级技术方案；

3. 所有数据库表、API 接口、字段命名遵循驼峰式/下划线统一规范，前后端协同确认。

2 用户需求说明

2.1 用户群体分析

2.1.1 目标用户

关注自身健康、希望改善生活习惯的**亚健康人群、健身小白、职场上班族**，具备基础智能手机/电脑操作能力，无专业健康管理知识，追求「轻量化、无负担」的健康记录方式。

2.1.2 核心用户痛点

1. 传统健康软件需精确录入数值（如卡路里、运动距离），操作繁琐，难以坚持；
2. 健康建议模板化，未结合用户历史行为、身体状况与目标，缺乏针对性；
3. 断签后数据断层，AI 建议出现偏差甚至错误，导致功能失效；
4. 仅展示冰冷数据，无可视化趋势与情绪价值引导，用户易产生任务疲劳与挫败感；
5. 部分工具过度医疗化，或存在 AI 越界给出医疗建议的安全风险。

2.2 核心业务需求（User Stories）

基于 Scrum 敏捷开发框架，将用户需求拆解为 3 个核心 Epic，涵盖**数据采集、智能分析、闭环管理**全流程，所有 Story 均为「必须实现」级别：

Epic 1：多模态极简健康数据采集

- **Story 1.1**: 作为新用户，我希望完成基础健康档案填写（目标、病史、身体指标），让 AI 在无历史数据时也能给出安全的初始建议，避免「冷启动」无意义回复；
- **Story 1.2**: 作为用户，我希望通过自然语言输入（如「跑了 40 分钟，吃了一个苹果」）记录健康数据，系统自动提取特征与估算数值，无需手动查卡路里/填表单；
- **Story 1.3**: 作为用户，我希望对 AI 解析的模糊数据进行修改（如纠正苹果的热量估算），确保数据准确性，避免后续建议基于错误数据叠加；
- **Story 1.4**: 作为用户，我希望系统保留我的原始输入内容，让 AI 的分析建

议更具「人情味」（如提及「你这周吃了三次苹果」），而非仅基于冰冷数值。

Epic 2：个性化连续式 AI 健康分析与建议

- **Story 2.1**: 作为用户，我希望 AI 给出建议时结合我的近 7 天数据+长期健康趋势，能说出「相比上个月，你的耐力明显提升」，避免每次像对待陌生人；
- **Story 2.2**：作为用户，若我连续 3 天未记录数据，希望 AI 给出「欢迎回归」的恢复性建议，而非追究未完成项或误判数据，避免断签后功能失效；
- **Story 2.3**: 作为用户，若我前一天任务完成率为 0%，希望 AI 降低次日任务难度并给予安慰，避免产生挫败感而放弃使用；
- **Story 2.4**: 作为用户，我希望系统以周/月为单位生成可视化图表（折线图/雷达图），直观展示我的热量、睡眠、运动趋势，清晰感知健康变化。

Epic 3：健康管理闭环与任务打卡

- **Story 3.1**: 作为用户，我希望 AI 生成的健康建议自动转化为结构化待办任务，支持一键打卡，无需手动创建计划；
- **Story 3.2**: 作为用户，我希望修改健康目标（如从减脂切换为增肌）后，AI 能重新规划建议，忽略与新目标冲突的历史饮食/运动偏好；
- **Story 3.3**: 作为用户，我希望打开 App 时能立即看到今日建议，无需等待 AI 实时生成，提升使用体验；
- **Story 3.4**: 作为用户，若我输入胸痛、晕厥等不适症状，希望系统及时提醒我就医，而非给出健康建议，避免延误医疗。

2.3 核心业务流程

系统采用**带泳道的跨职能流程**，分为**前端展示层、后端逻辑层、AI 服务层、数据存储层**，包含**正常处理分支**与**异常处理分支**，全程实现闭环管理，流程步骤如下：

1. **起点**：用户登录前端，完成基础健康档案填写（新用户）或录入健康数据（自然语言/手动）；
2. **安全拦截层**：后端对用户输入进行高危医疗词汇正则匹配，若触发则直接返回「立即就医警告」，阻断后续流程；未触发则进入下一步；
3. **特征提取层**：后端调用 LLM 将用户模糊输入转化为**结构化 JSON（估算数值+健康标签）**，前端展示解析结果并允许用户修改；用户确认后，系统以”原始输入+解析数据+健康标签“双轨制存入数据存储层；

4. **上下文组装层**：后端从数据存储层拉取用户**基础档案、近 7 天详细数据、历史周总结（长期记忆）、昨日任务完成率、当前健康目标**，组装为带约束的 Prompt；

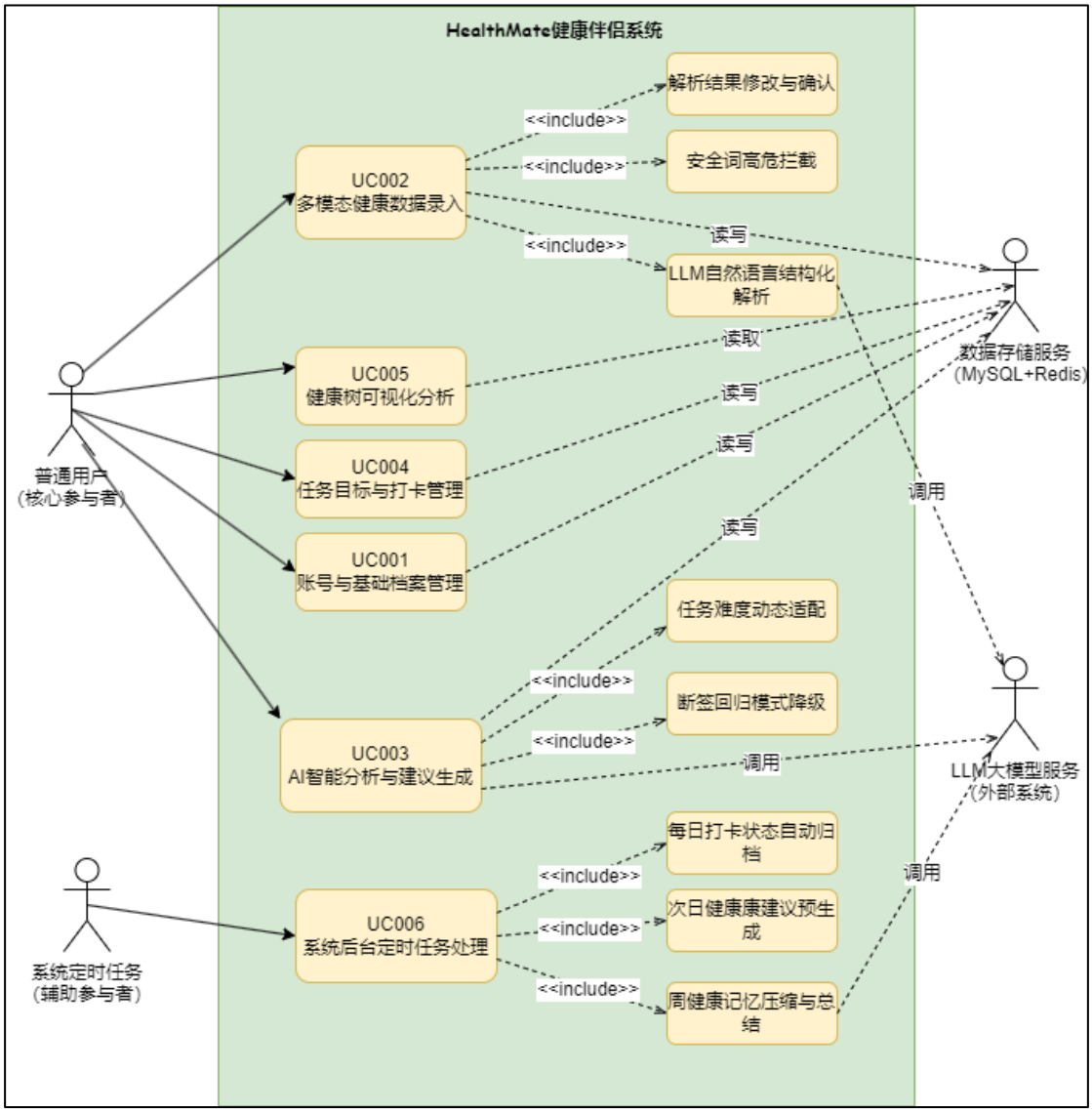
5. **AI 分析层**：后端调用 LLM 大模型 API，若 API 宕机/超时则触发 **Mock 本地兜底**，返回预置建议；正常响应则生成自然语言建议与结构化任务 JSON；

6. **数据落地层**：后端将 AI 生成的建议与任务 JSON 存入数据存储层，同时触发**定时任务**：每周生成健康周总结（压缩长期记忆）、每日凌晨预生成次日健康建议；

7. **终点**：前端以流式输出渲染 AI 建议，展示结构化待办任务列表与健康数据可视化图表；用户完成任务打卡后，打卡状态回传数据存储层，形成记录-分析-建议-打卡-反馈完整闭环。

3 需求分析建模

3.1 用例建模（Use Case Diagram）



3.1.1 参与者定义

1. 核心参与者：普通用户（User）—— 执行所有前端交互功能，对应功能需求规格中 4.1-4.5 模块；

2. 辅助参与者：系统定时任务（System Task）—— 执行后台自动化功能，对应功能需求规格中 4.6 模块。

3.1.2 核心用例详情

1. 账号与基础档案管理（UC001）

对应功能模块：4.1 账号与基础档案管理模块

前置条件：

1. 用户未登录时，已进入注册/登录页面；
2. 已登录用户未填写基础档案，或需修改档案信息；
3. 系统 JWT 鉴权模块正常运行。

基本流程：

1. 用户输入账号密码完成注册/登录，后端通过 JWT 鉴权生成 Token；
2. 新用户强制跳转至基础档案填写页，填写性别、身高、体重、健康目标、伤病史，提交后存入 t_user 表；
3. 已登录用户可进入「个人中心」，修改健康目标、伤病史等信息，修改后同步更新 t_user 表的 health_goal 与 health_goal_version 字段；
4. 前端弹窗提示「信息更新成功，AI 将重新规划建议」，流程结束。

异常流程：

1. 注册时用户名已存在，后端返回「用户名已被占用」，前端提示用户更换用户名后重新提交；
2. 填写档案时未勾选健康目标（必填项），前端表单校验拦截，提示「请选择健康目标」；
3. 修改信息时网络中断，前端提示「网络异常，信息未保存」，缓存输入内容，重新联网后可继续提交。

后置条件：

1. 注册/登录成功后，用户获得系统访问权限；
2. 档案信息存入/更新至数据库，为 AI 建议提供基础约束；
3. Token 有效期 2 小时，支持后续接口调用。

2. 多模态健康数据录入（UC002）

对应功能模块：4.2 多模态健康数据录入模块

前置条件：

1. 用户已登录并通过 JWT 鉴权；
2. 系统 LLM 解析模块、安全词汇校验模块正常运行；
3. 前端输入框/语音录入功能可用。

基本流程：

1. 用户选择「自然语言录入」或「手动表单录入」，输入健康相关数据（如「跑 40 分钟，吃 1 个苹果」或手动填写睡眠 8 小时）；
2. 后端触发安全词汇校验，未检测到高危词汇则继续；
3. 自然语言录入时，后端调用 LLM 解析为结构化数据（估算热量、健康标签

等），前端以「预填写卡片」展示；
4. 用户核对解析结果，可修改数值/删除标签，确认后系统以「原始输入+解析数据」双轨制存入 t_health_record 表；
5. 手动录入时，数据直接校验后存库，流程结束。

异常流程：
1. 输入包含「胸痛、晕厥」等高危词汇，后端拦截并返回「检测到高危症状，请立即就医」，阻断存库；
2. LLM 解析失败（输入无健康相关信息），前端提示「无法识别健康数据，请重新输入」；
3. 手动录入时睡眠时长填写「30 小时」（超出合理范围），前端校验提示「数值异常，建议输入 0-24 小时」。

后置条件：
1. 健康数据存入 t_health_record 表，生成唯一 record_id；
2. 数据同步至健康仪表盘，支撑后续趋势分析；
3. 原始输入保留，为 AI 建议提供「人情味」依据。

3. AI 智能分析与建议（UC003）

对应功能模块：4.3 AI 智能分析与建议模块

前置条件：
1. 用户已登录并通过鉴权；
2. t_user 表有用户基础档案，t_health_record 表有至少 1 条健康数据；
3. 系统 Redis 缓存、LLM API（或 Mock 模块）正常运行。

基本流程：
1. 用户打开 App，前端从 Redis 缓存读取预生成的今日建议（每日凌晨 4 点定时任务生成）；
2. 后端通过 SSE 流式输出建议至前端，实现「打字机」效果；
3. AI 建议自动拆解为 1-3 个轻量化任务，存入 t_daily_task 表，前端展示任务列表及 ai_reason；
4. 若用户连续≥2 天未记录数据，建议切换为「回归模式」；若昨日完成率 0%，任务难度降低，流程结束。

异常流程：
1. Redis 缓存无预生成建议，后端实时调用 LLM，前端提示「正在生成建议，请稍候」；
2. LLM API 宕机/超时（5 秒超时），触发 Mock 兜底，返回本地预置建议；
3. 数据量不足（仅 1 条健康记录），建议提示「数据较少，建议多记录几天以获得更精准分析」。

后置条件：

1. AI 建议与任务存入数据库，支持用户查看；
2. 建议内容包含长期记忆（上周总结）与短期数据，确保连续性；
3. 任务难度适配用户历史完成率。

4. 任务目标与打卡管理（UC004）

对应功能模块：4.4 任务目标与打卡管理模块

前置条件：

1. 用户已登录，t_daily_task 表有当日未完成任务；
2. 系统数据库事务模块正常运行。

基本流程：

1. 前端展示当日待办任务列表，标注任务内容、ai_reason；
2. 用户完成任务后点击「打卡」，前端向后端发起 POST /api/task/check 请求；
3. 后端开启事务，更新 t_daily_task 表的 status 为 1（已完成），同步计算当日完成率；
4. 事务提交成功，前端提示「打卡成功」，更新任务状态，流程结束。

异常流程：

1. 打卡时网络中断，前端提示「打卡失败」，缓存打卡操作，重新联网后自动重试；
2. 任务已过期（超出当日 24 点），点击打卡时提示「任务已过期，无法打卡」；
3. 数据库事务提交失败，后端返回「系统异常」，前端保持任务未完成状态，支持重新打卡。

后置条件：

1. 任务打卡状态同步至数据库；
2. 完成率数据更新，为次日任务难度调整提供依据；
3. 用户可在「历史记录」中查看打卡情况。

5. 健康数据可视化（UC005）

对应功能模块：4.5 健康数据可视化模块

前置条件：

1. 用户已登录，t_health_record 表有至少 7 条健康数据（支持周趋势）；
2. 前端 ECharts 组件加载正常。

基本流程：

1. 用户进入「健康仪表盘」，前端默认展示近 7 天睡眠、热量摄入/消耗折线图；
2. 用户可切换「周/月」维度，查看对应周期趋势图，支持鼠标悬浮查看具体数值；
3. 健康标签统计以饼图展示，标注「有氧、高糖」等标签占比；
4. 系统自动标注异常点（如睡眠＜6 小时），生成简单统计结论，流程结束。

异常流程：
1. 数据不足（＜7 条），周趋势图不显示，提示「数据不足，无法生成周趋势」；
2. ECharts 组件加载失败，前端展示纯文字统计（如「近 3 天平均睡眠 7.2 小时」）；
3. 切换月维度时网络卡顿，前端提示「加载中」，加载完成后刷新图表。

后置条件：
1. 用户成功查看健康数据可视化结果；
2. 异常点标注帮助用户快速感知健康问题；
3. 支持后续数据导出（优化功能）。

6. 系统后台定时任务（UC006）

对应功能模块：4.6 系统后台定时任务模块

前置条件：
1. 系统处于正常运行状态，无宕机；
2. 到达预设执行时间（每日 1 点/4 点、每周日 0 点）；
3. Celery 定时任务模块、LLM 压缩模块正常运行。

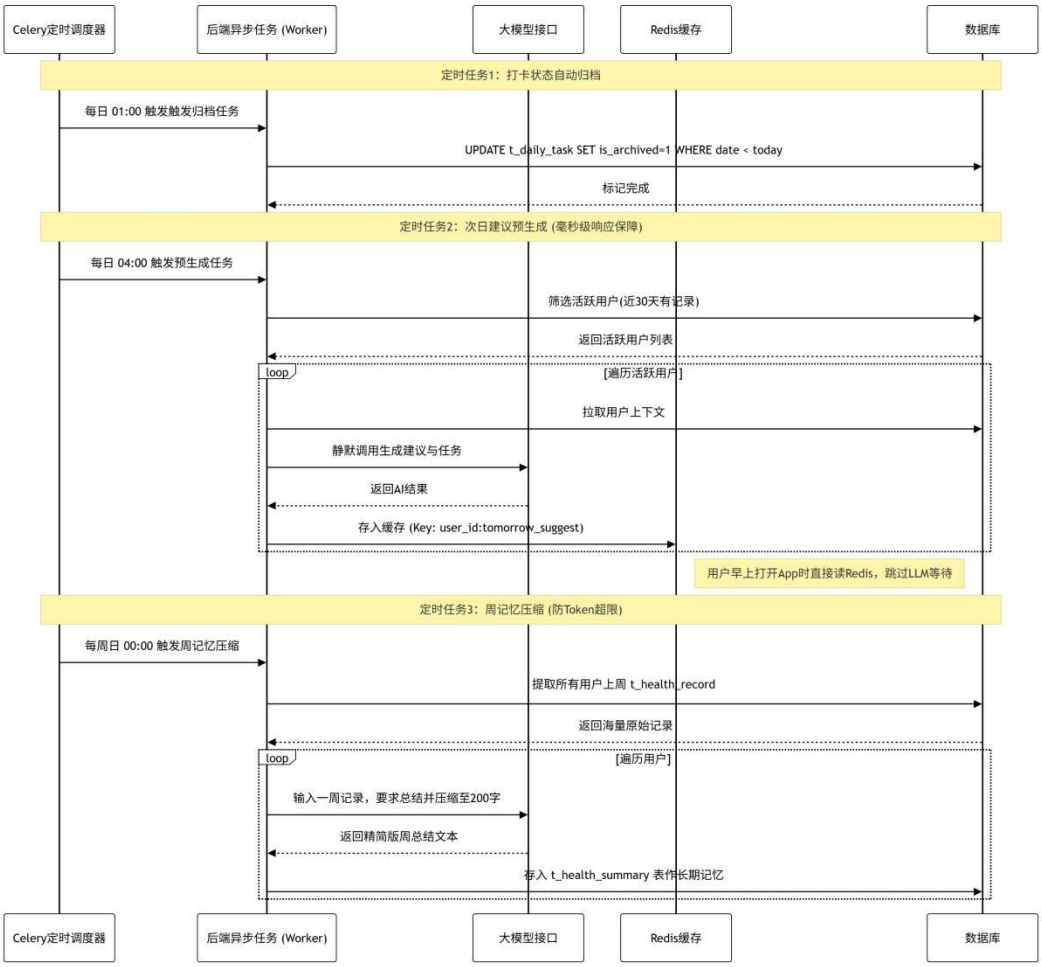
基本流程：
1. 每日凌晨 1 点：触发打卡状态归档任务，将前一日 t_daily_task 表数据标记为「已归档」，禁止修改；
2. 每日凌晨 4 点：触发次日建议预生成任务，为近 30 天有记录的活跃用户生成建议，存入 Redis 缓存；
3. 每周日 0 点：触发周记忆压缩任务，调用 LLM 将用户上周 t_health_record 数据压缩为 200 字内总结，存入 t_health_summary 表；
4. 任务执行完成，记录日志，流程结束。

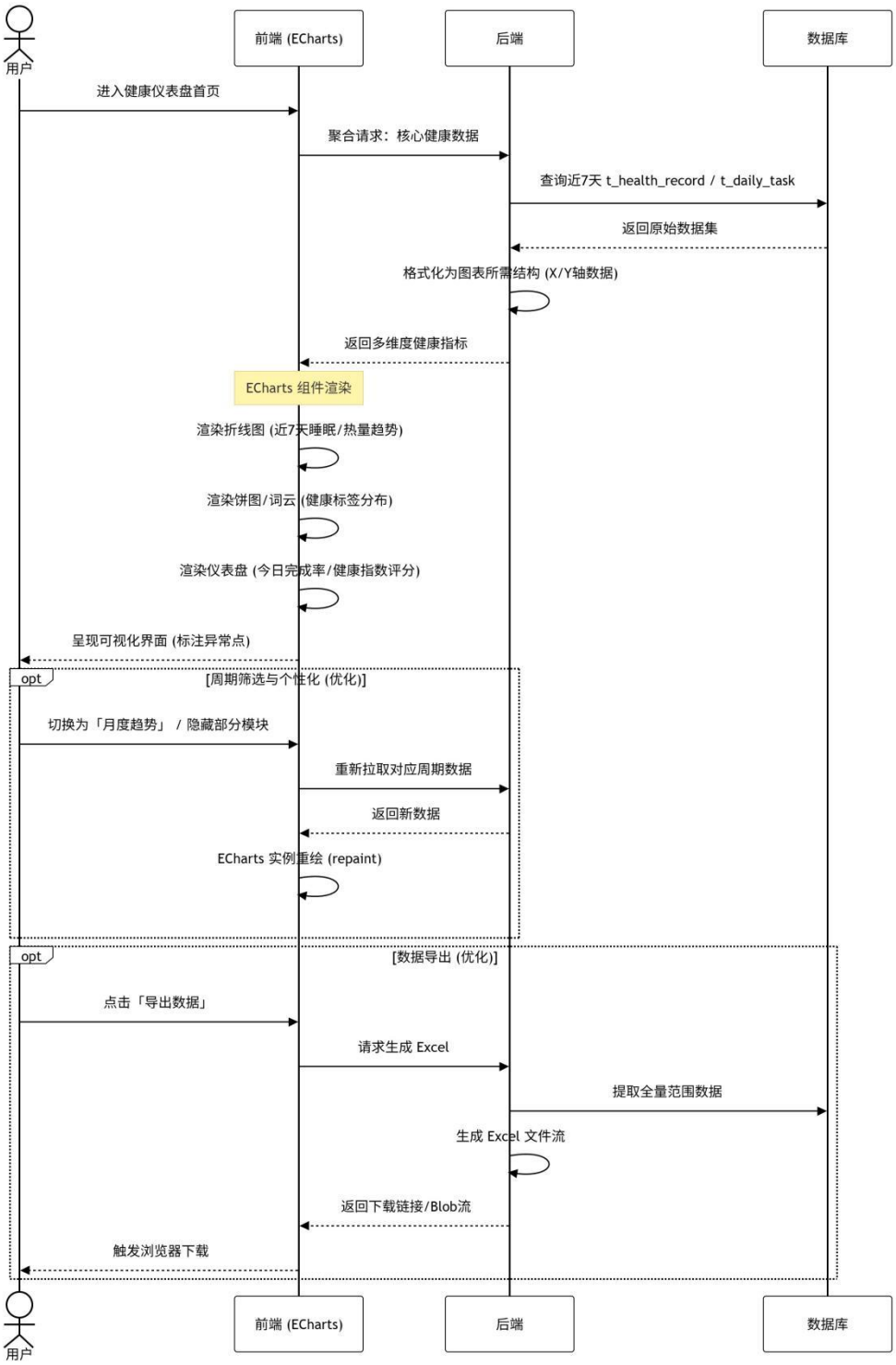
异常流程：
1. 定时任务执行失败，系统记录错误日志，次日同一时间自动重试；
2. 周总结压缩时部分用户数据异常，跳过该用户，记录异常日志，不影响其他用户；
3. Redis 缓存满，删除 7 天前的预生成建议，释放空间后继续存储新建议。

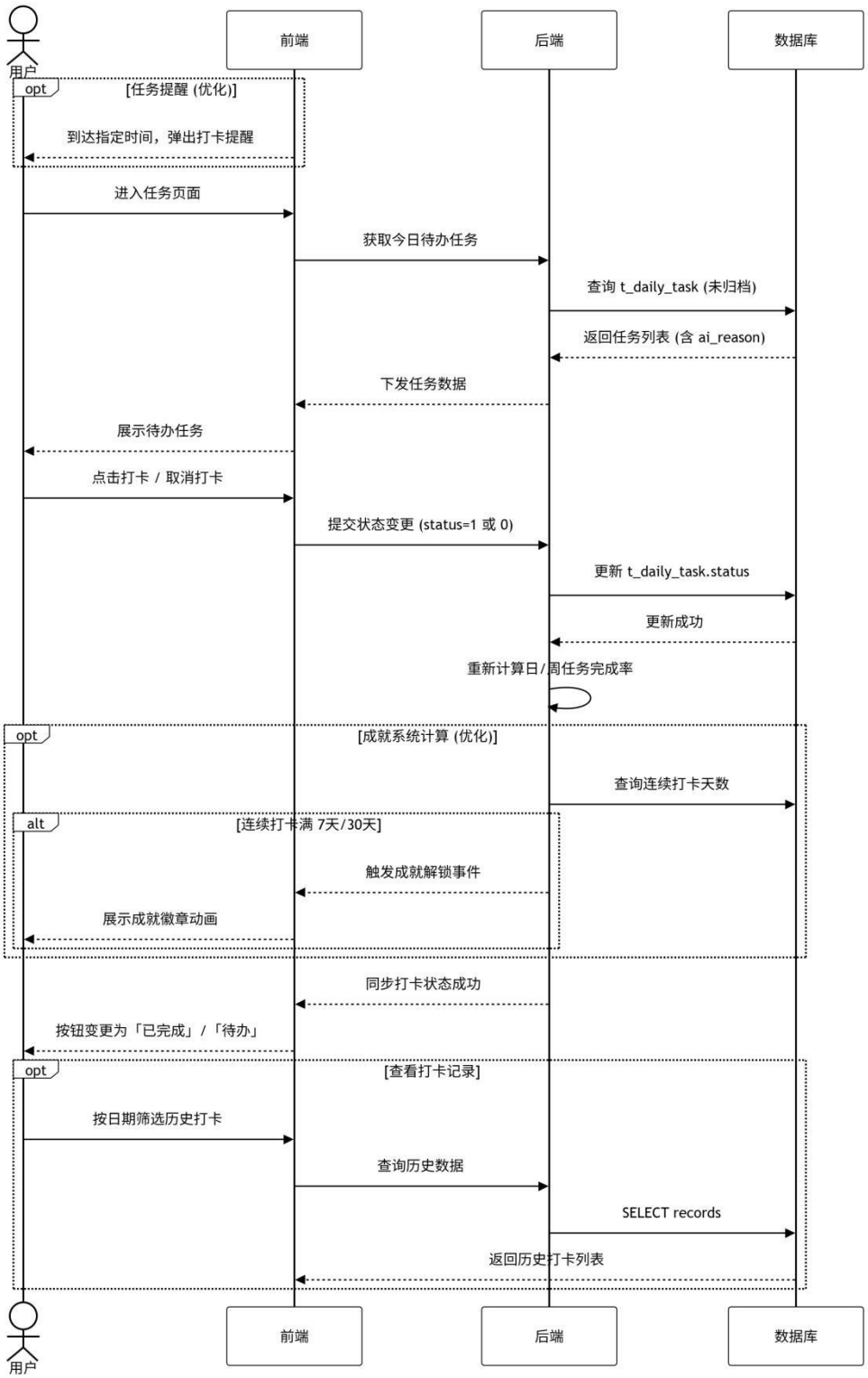
后置条件：

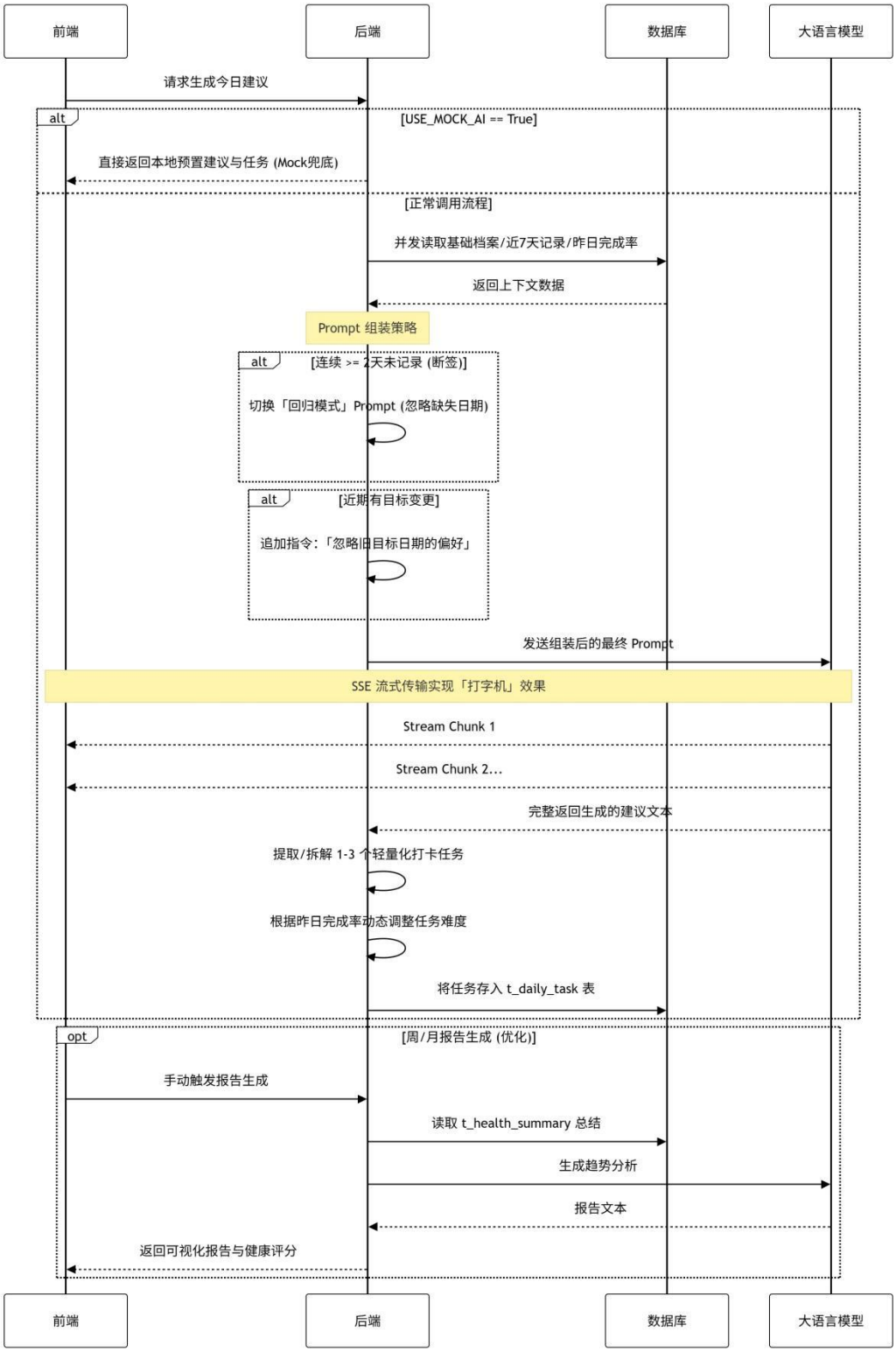
1. 打卡数据归档完成，确保数据一致性；
2. 次日建议存入缓存，提升用户访问速度；
3. 周总结生成，为 AI 长期记忆提供支撑。

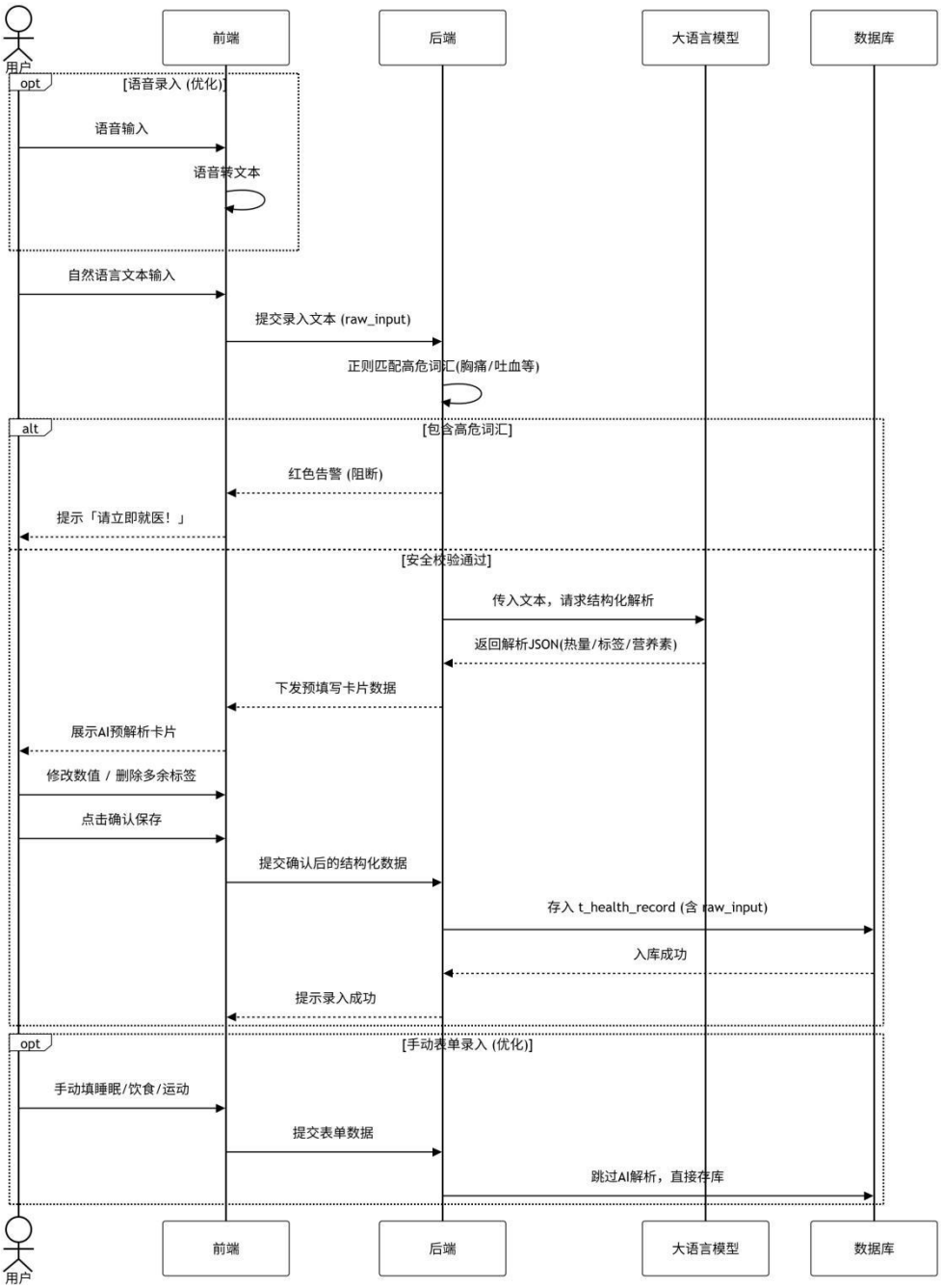
3.2 核心时序建模（Sequence Diagram）

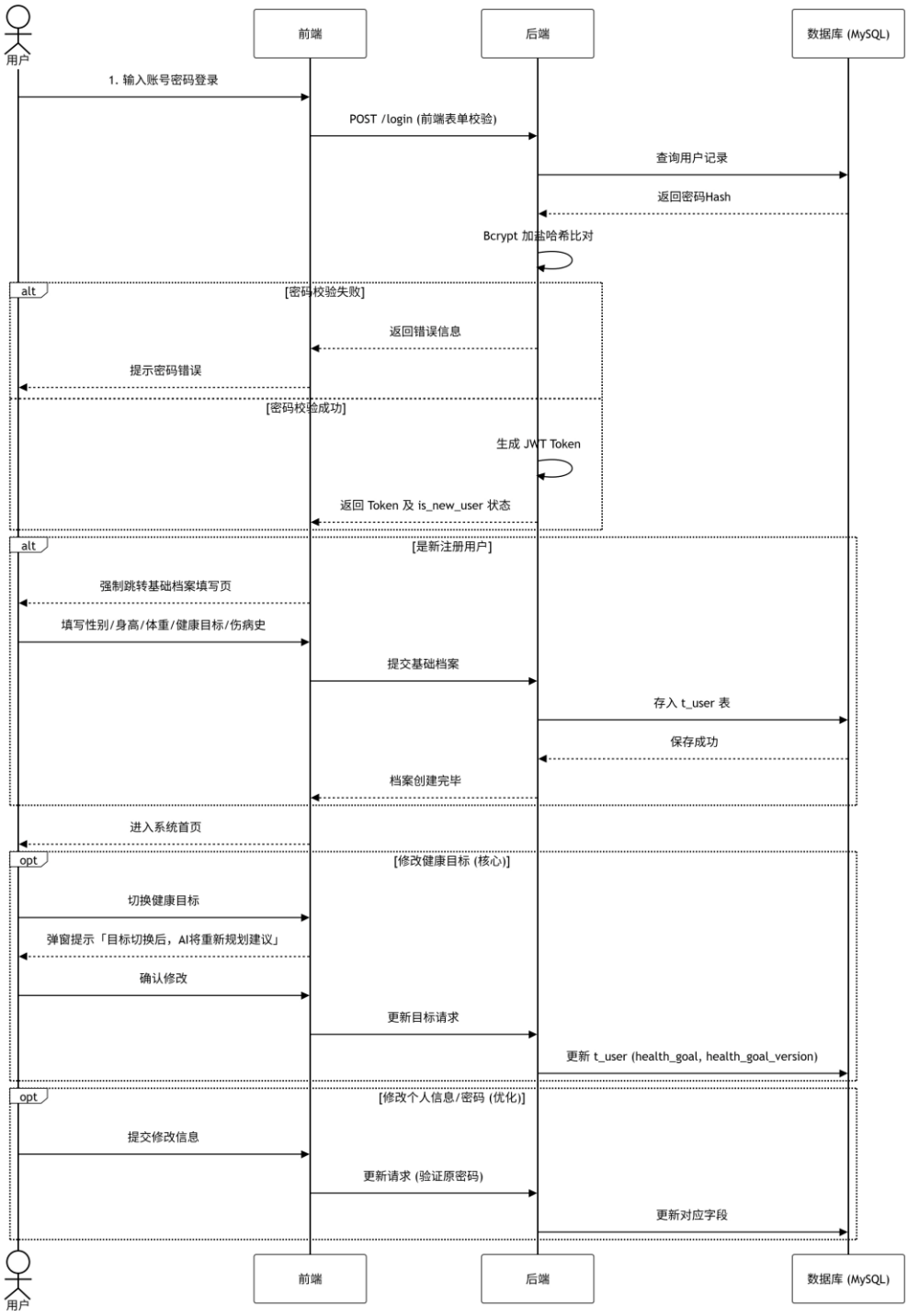



用户
前端 (ECharts)
后端
数据库
进入健康仪表盘首页
聚合请求：核心健康数据
查询近7天 t_health_record / t_daily_task
返回原始数据集
格式化为图表所需结构 (X/Y轴数据)
返回多维度健康指标
ECharts 组件渲染
渲染折线图 (近7天睡眠/热量趋势)
渲染饼图/词云 (健康标签分布)
渲染仪表盘 (今日完成率/健康指数评分)
呈现可视化界面 (标注异常点)
opt
[周期筛选与个性化 (优化)]
切换为「月度趋势」 / 隐藏部分模块
重新拉取对应周期数据
返回新数据
ECharts 实例重绘 (repaint)
opt
[数据导出 (优化)]
点击「导出数据」
请求生成 Excel
提取全量范围数据
生成 Excel 文件流
返回下载链接/Blob流
触发浏览器下载
用户
前端 (ECharts)
后端
数据库


用户
前端
后端
数据库
opt
[任务提醒 (优化)]
到达指定时间，弹出打卡提醒
进入任务页面
获取今日待办任务
查询 t_daily_task (未归档)
返回任务列表 (含 ai_reason)
下发任务数据
展示待办任务
点击打卡 / 取消打卡
提交状态变更 (status=1 或 0)
更新 t_daily_task.status
更新成功
重新计算日/周任务完成率
opt
[成就系统计算 (优化)]
查询连续打卡天数
alt
[连续打卡满 7天/30天]
触发成就解锁事件
展示成就徽章动画
同步打卡状态成功
按钮变更为「已完成」/「待办」
opt
[查看打卡记录]
按日期筛选历史打卡
查询历史数据
SELECT records
返回历史打卡列表
用户
前端
后端
数据库


前端
后端
数据库
大语言模型
请求生成今日建议
alt
[USE_MOCK_AI == True]
直接返回本地预置建议与任务 (Mock兜底)
[正常调用流程]
并发读取基础档案/近7天记录/昨日完成率
返回上下文数据
Prompt 组装策略
alt
[连续 >= 2天未记录 (断签)]
切换「回归模式」Prompt (忽略缺失日期)
alt
[近期有目标变更]
追加指令：「忽略旧目标日期的偏好」
发送组装后的最终 Prompt
SSE 流式传输实现「打字机」效果
Stream Chunk 1
Stream Chunk 2...
完整返回生成的建议文本
提取/拆解 1-3 个轻量化打卡任务
根据昨日完成率动态调整任务难度
将任务存入 t_daily_task 表
opt
[周/月报告生成 (优化)]
手动触发报告生成
读取 t_health_summary 总结
生成趋势分析
报告文本
返回可视化报告与健康评分
前端
后端
数据库
大语言模型


用户
前端
后端
大语言模型
数据库
opt
[语音录入 (优化)]
语音输入
语音转文本
自然语言文本输入
提交录入文本 (raw_input)
正则匹配高危词汇(胸痛/吐血等)
alt
[包含高危词汇]
红色告警 (阻断)
提示「请立即就医！」
[安全校验通过]
传入文本，请求结构化解析
返回解析JSON(热量/标签/营养素)
下发预填写卡片数据
展示AI预解析卡片
修改数值 / 删除多余标签
点击确认保存
提交确认后的结构化数据
存入 t_health_record (含 raw_input)
入库成功
提示录入成功
opt
[手动表单录入 (优化)]
手动填睡眠/饮食/运动
提交表单数据
跳过AI解析，直接存库
用户
前端
后端
大语言模型
数据库



以获取 AI 今日健康建议并生成打卡任务为核心场景，展示系统各模块的消息调用顺序与交互逻辑，参与对象包括**用户、前端应用（React）、后端服务（FastAPI）、MySQL 数据库、Redis 缓存、LLM 大模型 API**，时序步骤如下：

1. 用户点击前端「生成今日健康建议」按钮；
2. 前端通过 JWT 鉴权后，向后端发起 GET /api/health/daily-report 请求；
3. 后端执行高危医疗词汇校验（本地正则匹配），若触发则直接返回告警信息给前端，流程终止；

4. 后端从 MySQL 数据库拉取用户基础档案、近 7 天健康记录、昨日任务完成率、当前健康目标版本；

5. 后端从 Redis 缓存读取该用户的上周健康周总结（长期记忆），组装为带系统约束的 Prompt；

6. 后端判断环境变量 USE_MOCK_AI 状态：若为 True 则直接返回本地预置的 Mock 建议与任务；若为 False 则调用 LLM 大模型 API；

7. LLM 大模型以流式响应返回自然语言建议，后端通过 SSE（Server-Sent Events） 实时推送给前端，实现「打字机」效果；

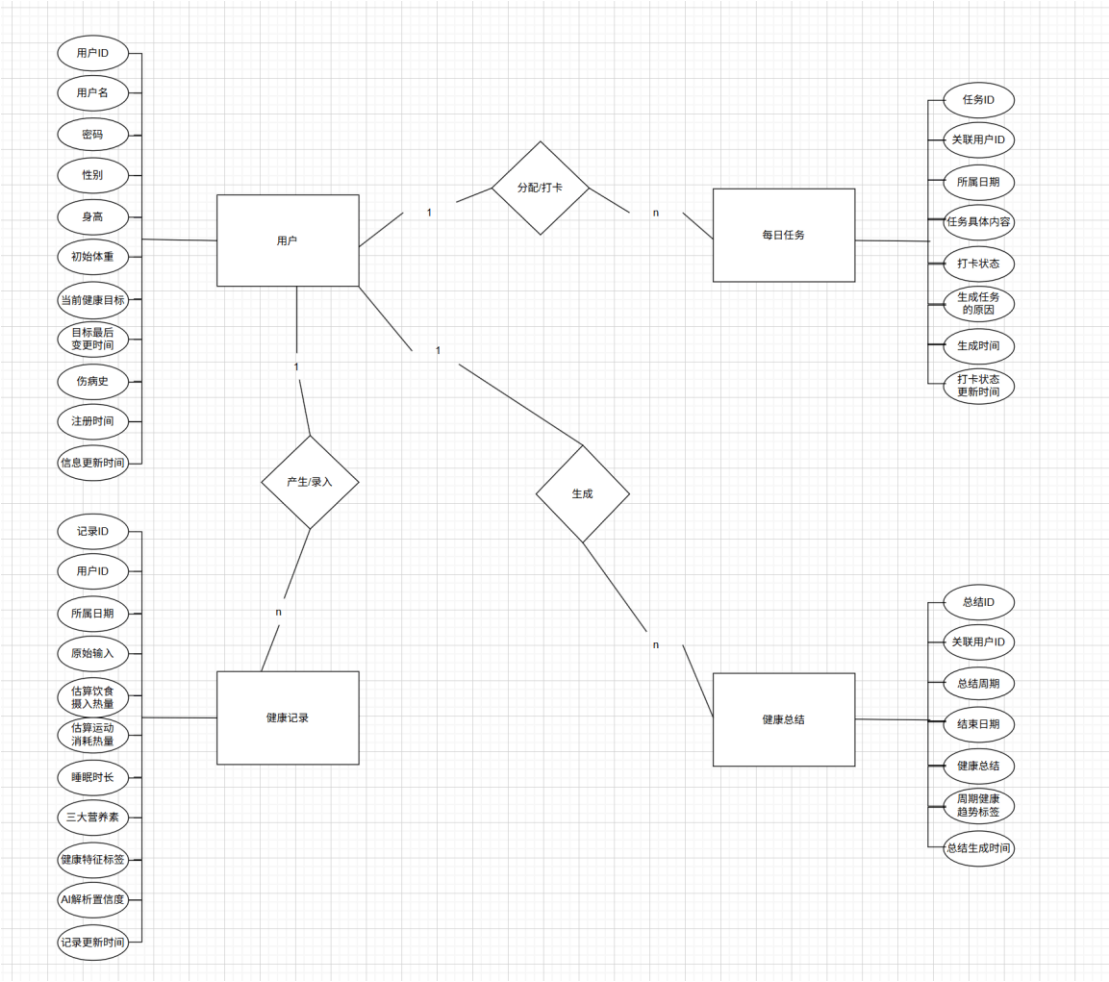
8. LLM 大模型追加返回结构化任务 JSON，后端解析后将任务存入 MySQL 的每日任务表；

9. 后端将健康建议与任务列表整合，返回给前端；

10. 前端渲染建议内容、待办任务列表与健康数据图表，展示给用户；

11. 用户完成任务打卡后，前端向后端发起 POST /api/task/check 请求，后端更新 MySQL 中任务的打卡状态，流程结束。

3.3 数据建模（ER 图）



采用**关系型数据库+JSON 字段**的混合设计，解决「营养素细化导致数据库字段爆炸」「健康标签灵活扩展」的问题，包含 3 **个核心实体**，所有实体关系均为**一对多**（1:N），实体与属性设计如下：

实体 1：用户（t_user）

- 核心属性：user_id（PK，自增）、username、password_hash、gender、height、weight、health_goal（当前健康目标）、health_goal_version（目标版本，记录变更时间）、medical_history（伤病史/过敏史）、created_at、updated_at;
- 核心作用：存储用户基础信息与健康档案，为 AI 建议提供**安全约束与个性化基准**。

实体 2：健康记录流水（t_health_record）

- 核心属性：record_id（PK，自增）、user_id（FK，关联 t_user）、record_date、raw_input（用户原始自然语言输入）、estimated_intake_kcal（估算饮食热量）、estimated_burn_kcal（估算运动热量）、sleep_minutes（睡眠时长）、nutrition_details（JSON，存储三大宏量营养素）、health_tags（JSON，存储健康特

征标签）、confidence（AI 解析置信度）、updated_at；

- 核心作用：以**双轨制**存储用户健康数据，保留原始输入提升建议人情味，通过 JSON 字段实现标签化与营养素存储，避免表结构臃肿。

实体 3：每日任务（t_daily_task）

- 核心属性：task_id（PK，自增）、user_id（FK，关联 t_user）、task_date、task_content（任务内容）、status（打卡状态：0 未完成/1 已完成）、ai_reason（AI 生成任务的缘由）、created_at、updated_at；
- 核心作用：存储 AI 生成的结构化打卡任务，记录打卡状态，为后续任务难度调整提供**反馈数据**。

实体 4：健康总结（t_health_summary）

- 核心属性：summary_id（PK，自增）、user_id（FK，关联 t_user）、summary_cycle（周期：周/月）、summary_date（周期结束日期）、summary_content（AI 生成的总结内容，200 字以内）、health_trend（JSON，存储周期健康趋势标签）、created_at；
- 核心作用：存储用户健康周/月总结，实现**记忆压缩**，为 AI 提供长期健康趋势，同时控制 LLM Prompt 的 Token 消耗。

3.4 核心数据字典

表 1：用户基础信息表（t_user）

字段名	数据类型	约束	非空	描述说明
user_id	INT	PK, AUTO_INCREMENT	是	用户唯一标识
username	VARCHAR(50)	UNIQUE	是	用户登录名
password_hash	VARCHAR(255)	-	是	密码 bcrypt 加盐哈希值，禁止明文存储
gender	TINYINT	-	否	性别：0 未知/1 男/2 女
height	DECIMAL(5,2)	-	否	身高（cm）
weight	DECIMAL(5,2)	-	否	初始体重（kg）
health_goal	VARCHAR(50)	-	是	当前健康目标：减脂/增肌/改善睡眠/保持健康
health_goal_version	DATETIME	-	是	健康目标最后变更时间，用于区分新旧目标

表一（续表）

字段名	数据类型	约束	非空	描述说明
medical_history	VARCHAR(255)	-	否	伤病史/过敏史，AI 建议安全约束（如膝盖损伤）
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	是	注册时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	是	信息更新时间

表 2：健康记录流水表（t_health_record）【双轨制+JSON 设计】

字段名	数据类型	约束	非空	描述说明
record_id	INT	PK, AUTO_INCREMENT	是	记录唯一标识
user_id	INT	FK → t_user	是	关联用户 ID
record_date	DATE	-	是	记录归属日期（YYYY-MM-DD）
raw_input	VARCHAR(500)	-	否	用户原始自然语言输入（如「跑了 40 分钟，吃了一个苹果」）
estimated_intake_kcal	INT	-	否	AI 估算饮食摄入热量（千卡）
estimated_burn_kcal	INT	-	否	AI 估算运动消耗热量（千卡）
sleep_minutes	INT	-	否	睡眠总时长（分钟）
nutrition_details	JSON	-	否	三大宏量营养素 JSON: {"carbs_g":x, "protein_g":x, "fat_g":x}

表二（续表）

字段名	数据类型	约束	非空	描述说明
health_tags	JSON	-	否	健康特征标签 JSON 数组：["有氧", "高糖", "缺乏运动"]
confidence	VARCHAR(10)	-	否	AI 解析置信度：low/medium/high
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	是	记录更新时间

表 3：每日打卡任务表（t_daily_task）

字段名	数据类型	约束	非空	描述说明
task_id	INT	PK, AUTO_INCREMENT	是	任务唯一标识
user_id	INT	FK → t_user	是	关联用户 ID
task_date	DATE	-	是	任务所属日期（YYYY-MM-DD）
task_content	VARCHAR(255)	-	是	任务具体内容（如「饭后散步 15 分钟，喝 2L 水」）
status	TINYINT	DEFAULT 0	是	打卡状态：0 未完成/1 已完成
ai_reason	TEXT	-	否	AI 生成该任务的缘由（如「昨日饮食高钠，建议多喝水」）
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	是	任务生成时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	是	打卡状态更新时间

表 4：健康总结表（t_health_summary）

字段名	数据类型	约束	非空	描述说明
summary_id	INT	PK, AUTO_INCREMENT	是	总结唯一标识
user_id	INT	FK → t_user	是	关联用户 ID
summary_cycle	VARCHAR(10)	-	是	总结周期：week（周）/month（月）
summary_date	DATE	-	是	周期结束日期（如周日/月末日期，YYYY-MM-DD）
summary_content	VARCHAR(200)	-	是	AI 生成的健康总结（200 字以内，压缩长期记忆）
health_trend	JSON	-	否	周期健康趋势标签 JSON 数组：["有氧运动不足", "饮食高碳水"]
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	是	总结生成时间

4 功能需求规格

按「用户操作流程+系统后台能力」分为 6 大功能模块，所有模块均为「必须实现」，模块下的子功能标注「核心」/「优化」，优先开发核心子功能。

4.1 账号与基础档案管理模块

核心子功能

1. **用户注册/登录**：支持账号密码登录，密码 bcrypt 加盐哈希存储，前端做表单校验，后端做 JWT 鉴权，返回 Token 用于后续接口调用；
2. **基础健康档案填写**：新用户注册后强制进入，填写性别、身高、体重、健康目标（单选）、伤病史/过敏史（文本），提交后存入 t_user 表；
3. **健康目标修改**：支持用户随时修改健康目标，修改后自动更新 t_user 表的 health_goal 与 health_goal_version 字段，前端弹窗提示「目标切换后，AI 将重新规划建议」。

优化子功能

1. **个人信息编辑**：支持修改身高、体重、伤病史，实时同步至数据库；
2. **密码修改**：验证原密码后，更新 password_hash 字段。

4.2 多模态健康数据录入模块

核心子功能

1. **自然语言录入**：前端提供输入框，支持用户输入任意健康相关文本，提交后后端调用 LLM 解析为结构化数据，返回前端展示；
2. **AI 解析结果修改**：前端以「预填写卡片」展示 AI 解析的热量、标签、营养素等信息，支持用户点击修改数值/删除标签，确认后双轨制存入 t_health_record 表；
3. **原始输入保留**：用户所有自然语言输入均存入 t_health_record 表的 raw_input 字段，不可修改，为 AI 分析提供上下文；
4. **安全词汇校验**：后端对用户输入进行正则匹配，包含「胸痛、吐血、晕厥、抽搐」等高危医疗词汇时，直接返回红色告警，阻断解析与存库。

优化子功能

1. **手动表单录入**：支持用户手动填写睡眠时长、饮食热量、运动热量，提交后直接存库，无需 AI 解析；

2. **语音录入**：前端集成语音识别，将语音转化为文本后，按自然语言录入流程处理。

4.3 AI 智能分析与建议模块

核心子功能

1. **今日建议生成**：结合用户基础档案、近 7 天记录、上周总结、昨日完成率，生成个性化自然语言建议，通过 SSE 流式输出至前端，实现「打字机」效果；

2. **建议结构化转化**：AI 建议自动拆解为 1-3 个轻量化打卡任务，存入 t_daily_task 表，任务数量根据用户历史完成率动态调整（完成率低则减少任务）；

3. **长期记忆管理**：每周日凌晨触发后台定时任务，将用户上周的 t_health_record 数据压缩为 200 字以内的周总结，存入 t_health_summary 表，作为长期记忆；

4. **断签优雅降级**：若用户连续≥2 天未记录数据，Prompt 自动切换为「回归模式」，仅基于用户基础档案与最新录入数据生成恢复性建议，忽略缺失日期；

5. **任务难度动态调整**：根据用户昨日任务完成率调整次日任务难度：完成率 0%则大幅降低难度（如跑步改散步），完成率≥80%则适当提升难度；

6. **目标变更适配**：组装 Prompt 时明确告知 AI「用户自[日期]起目标变更为[新目标]，忽略该日期前的饮食/运动偏好」，避免建议与新目标冲突；

7. **Mock 兜底**：后端提供全局环境变量 USE_MOCK_AI，开启后跳过 LLM API 调用，直接返回本地预置的建议与任务，确保演示/API 宕机时系统可用。

优化子功能

1. **周/月报告生成**：支持用户手动触发，AI 基于 t_health_summary 表的周/月总结，生成可视化的健康趋势报告；

2. **健康指数评分**：基于用户的饮食、运动、睡眠数据，生成 0-100 分的健康指数，实时展示在前端仪表盘。

4.4 任务目标与打卡管理模块

核心子功能

1. **待办任务展示**：前端按日期展示 t_daily_task 表中的未完成任务，标注任务缘由（ai_reason）；

2. **任务打卡/取消**：用户点击打卡按钮，后端更新 t_daily_task 表的 status 字段为 1；支持取消打卡，恢复为 0，打卡状态实时同步；

3. **任务完成率统计**：后端按日/周统计用户的任务完成率（完成任务数/总任务数），为 AI 调整任务难度提供数据；

4. **打卡记录查看**：支持用户查看历史日期的打卡任务与完成状态，按日期筛选。

优化子功能

1. **任务提醒**：前端提供每日打卡提醒，支持设置提醒时间；

2. **成就系统**：连续打卡 7 天/30 天生成成就标识，展示在用户个人中心，提升用户粘性。

4.5 健康数据可视化模块

核心子功能

1. **健康仪表盘**：前端首页展示核心健康数据：今日任务完成率、近 7 天睡眠时长趋势、近 7 天热量摄入/消耗对比，采用 ECharts 实现折线图；

2. **周/月趋势分析**：支持按周/月筛选，展示睡眠、热量、运动的趋势折线图，对比历史均值，标注异常点（如睡眠不足、热量超标）；

3. **健康标签统计**：展示用户近期的健康标签分布（如高糖、有氧、缺乏运动），采用饼图/词云展示，直观反映用户健康习惯。

优化子功能

1. **健康数据导出**：支持将周/月健康数据导出为 Excel 文件，方便用户本地查看；

2. **个性化仪表盘定制**：支持用户自主选择展示的数据分析模块，如隐藏热量趋势、优先展示睡眠数据。

4.6 系统后台定时任务模块

核心子功能【用户无感知，后端实现】

1. **周记忆压缩**：每周日凌晨 0 点，触发 Celery 定时任务，调用 LLM 将用户上周的 t_health_record 数据压缩为周总结，存入 t_health_summary 表；

2. **次日建议预生成**：每日凌晨 4 点，触发定时任务，为活跃用户（近 30 天有记录）预生成次日健康建议与任务，存入 Redis 缓存，用户打开 App 时直接读取，实现毫秒级响应；

3. **打卡状态自动归档**：每日凌晨 1 点，将前一日的 t_daily_task 表数据标记为「已归档」，禁止修改，确保数据一致性。

5 非功能性需求

所有非功能性需求均为「必须实现」，是保障系统**可用性、性能、安全性、可维护性**的核心，分为 6 大类：

5.1 性能需求

1. 页面响应时间：前端静态页面切换响应时间≤1 秒，动态数据（如打卡、修改解析结果）提交后响应时间≤500 毫秒；

2. API 响应时间：后端常规增删改查 API（如打卡、查记录）响应时间≤500 毫秒；AI 建议生成 API（流式输出）首字符响应时间≤3 秒，全程响应时间≤10 秒；

3. 并发处理能力：系统支持至少 50 个并发用户访问，无明显延迟；数据库支持至少 1000 条/天的数据写入，查询无卡顿；

4. 缓存性能：Redis 缓存的健康周总结、预生成建议，过期时间设置为 7 天，缓存命中率≥90%。

5.2 安全需求

1. 接口鉴权：所有后端 API 均需 JWT 鉴权，Token 有效期为 2 小时，过期后需重新登录；非法 Token 请求直接返回 401 未授权；

2. 密码安全：数据库禁止明文存储密码，必须使用 bcrypt 算法加盐哈希，盐值随机生成；前端做密码强度校验（至少 6 位，包含字母/数字）；

3. 数据脱敏：向 LLM API 发送的所有数据均做脱敏处理，仅传递用户的健康数据与标签，禁止发送用户名、user_id 等身份标识（PII）；

4. SQL 注入防护：后端使用 SQLAlchemy ORM 框架，避免原生 SQL 拼接，防止 SQL 注入；

5. XSS 防护：前端对用户输入的文本做转义处理，防止跨站脚本攻击；后端对接口入参做格式校验，拒绝非法字符。

5.3 可用性需求

1. 界面易用性：前端界面采用卡片式设计，操作流程简洁，无冗余步骤；核心功能（录入数据、打卡）支持一键操作，新用户无需学习即可使用；

2. 错误提示：系统所有错误均需给出清晰的中文提示，如「用户名已存在」「Token 过期，请重新登录」「检测到高危症状，请立即就医」，避免技术化

提示；

3. 断点续传：AI 建议流式输出过程中，若网络中断，重新连接后可继续输出，无需重新生成；

4. 跨设备适配：前端支持主流现代浏览器（Chrome≥90、Edge≥90、Safari≥14），适配电脑、平板、手机等设备，响应式布局。

5.4 可靠性需求

1. 数据持久性：用户所有健康数据、打卡记录、总结数据均存入 MySQL 数据库，开启自动备份，每日凌晨 2 点备份一次，备份文件保留 30 天；

2. 系统容错性：LLM API 宕机/超时（超时时间设置为 5 秒）时，自动触发 Mock 兜底，确保核心功能可用；网络中断时，前端提示「网络异常，请检查后重试」，不崩溃；

3. 数据一致性：打卡状态、健康数据修改等操作，采用数据库事务处理，确保操作要么全部成功，要么全部回滚，避免数据脏读/幻读。

5.5 可维护性需求

1. 代码规范：前后端代码遵循行业规范，前端采用 ESLint，后端采用 PEP8，代码注释率≥30%，关键逻辑（如 Prompt 组装、容错处理）必须写详细注释；

2. 接口文档：后端使用 Swagger 自动生成 API 接口文档，包含接口地址、请求方式、入参、出参、错误码，实时同步开发进度；

3. 配置分离：系统所有配置（如 LLM API 密钥、数据库连接信息、环境变量）均与代码分离，存入配置文件，便于部署与修改；

4. 日志管理：后端记录详细的操作日志与错误日志，日志包含用户 ID、操作时间、接口地址、请求参数、错误信息，日志文件按天分割，保留 30 天。

5.6 可扩展性需求

1. 模块解耦：系统采用前后端分离架构，AI 服务、数据存储、定时任务为独立模块，模块间通过 API/消息通信，便于单独升级与替换；

2. 字段扩展：数据库采用 JSON 字段存储健康标签与营养素，无需修改表结构即可新增标签/营养素，便于后续功能扩展；

3. 第三方服务适配：LLM API 接入层做封装，支持无缝切换 DeepSeek、Kimi、智谱 GLM 等国内大模型 API，仅需修改配置文件；

4. 功能扩展：预留可穿戴设备数据同步、饮食图片识别、匿名社交排行榜等

功能接口，便于后续迭代开发。

6 系统容错与安全设计

本系统针对**健康管理类产品的核心痛点**与 AI 应用的**潜在风险**，设计了 6 **层容错与安全机制**，均为必须实现的核心设计，是系统落地的关键：

6.1 数据录入容错：模糊输入解析+用户修正权

1. 由 LLM 承担「模糊数据估算器」角色，将自然语言转化为结构化数据，无需用户精确录入，降低输入门槛；

2. 引入「确认与微调」机制，AI 解析结果前端展示后，允许用户手动修改，确认后再存库，避免 AI 幻觉/错误导致的后续建议偏差；

3. 数据库双轨制存储「原始输入+解析数据」，既保证数据准确性，又让 AI 建议更具人情味。

6.2 断签容错：优雅降级+有效样本趋势分析

1. 若用户连续≥2 天未记录数据，Prompt 自动切换为「回归模式」，生成恢复性建议，不追究未完成项，避免用户产生挫败感；

2. 发给 LLM 的数据为「带日期戳的稀疏数组」，明确告知 AI「忽略缺失日期，视为未记录而非零活动」，仅基于有效数据点分析趋势；

3. 设定健康建议「安全护栏」，Prompt 中强制约束「单日热量缺口≤500 千卡，单次运动时长≤90 分钟」，即使数据有偏差，也不会给出伤害身体的建议。

6.3 AI 服务容错：Mock 兜底+流式输出

1. 后端提供全局环境变量 USE_MOCK_AI，开启后跳过 LLM API 调用，直接返回本地预置的建议与任务，解决答辩演示/API 宕机/欠费的问题，体现高可用性设计；

2. 采用 SSE（Server-Sent Events）实现 AI 建议的流式输出，前端呈现「打字机」效果，缓解用户等待焦虑，避免因长时等待导致的页面关闭；

3. 为 LLM API 调用设置 5 秒超时时间，超时后自动触发 Mock 兜底，确保系统不卡死。

6.4 长期记忆容错：记忆压缩+Prompt 分层组装

1. 实现「健康总结」机制，每周将原始数据压缩为 200 字以内的周总结，存入专门的总结表，避免 LLM Prompt 因历史数据过多导致 Token 消耗过高、响应

变慢；

2. Prompt 采用「系统约束+长期记忆（上周总结）+短期记忆（近 7 天详细数据）+反馈数据（昨日完成率）」的分层组装策略，既保证建议的连续性，又控制 Token 成本。

6.5 任务管理容错：动态难度调整+目标版本管理

1. 基于用户昨日任务完成率动态调整次日任务难度，完成率低则降低难度、减少任务数量，同时给予情绪安慰，避免任务疲劳与挫败感；

2. 在用户表中引入「健康目标版本」字段，记录目标变更时间，组装 Prompt 时明确告知 AI 目标变更节点，忽略与新目标冲突的历史数据，避免建议逻辑冲突。

6.6 医疗安全容错：词汇拦截+Prompt 强约束

1. 后端正则拦截：在用户输入传递给 LLM 前，做高危医疗词汇匹配，触发后直接返回「立即就医警告」，阻断后续流程，是第一道安全防线；

2. Prompt 强约束：System Prompt 中强制声明「你是日常健康习惯监督员，非医生，禁止提供任何疾病诊断、处方药建议，对急性不适需建议就医」，是第二道安全防线；

3. 前端显性提示：App 首页/个人中心展示「免责声明」，明确标注「本产品为非医疗级健康管理工具，不替代专业医生诊断」，是第三道安全防线。

7 开发与部署环境

7.1 开发环境

前端开发环境

- 操作系统：Windows/macOS/Linux（兼容）；
- 开发工具：VS Code + ESLint + Prettier；
- 技术栈：React 18+、ECharts 5+、Ant Design Mobile/Element Plus（UI 组件）、Axios（网络请求）、SSE（流式输出）、JWT（鉴权）。

后端开发环境

- 操作系统：Linux（Ubuntu 20.04 LTS）/macOS；
- 开发工具：PyCharm + PEP8 + Swagger；
- 技术栈：Python 3.10+、FastAPI 0.100+、SQLAlchemy 2.0+（ORM）、Celery 5.2+（定时任务）、LangChain 0.1+（Prompt 编排）、JWT（鉴权）、bcrypt（密码加密）。

数据存储环境

- MySQL 8.0+：支持 JSON 字段、事务、自动备份；
- Redis 6.0+：用于缓存 JWT Token、健康总结、预生成建议，支持定时过期。

7.2 部署环境

服务器环境

- 云服务器：阿里云/腾讯云学生服务器（Linux Ubuntu 20.04 LTS，1 核 2G 以上）；
- 容器化：Docker 20.10+ + Docker Compose 2.0+，实现前后端、数据库、Redis 的容器化编排，确保开发/生产环境一致；
- 反向代理：Nginx 1.20+，负责前端静态资源托管、后端 API 反向代理、负载均衡。

第三方服务

- LLM API：DeepSeek API / 月之暗面 Kimi API / 智谱 GLM API（三选一，根据调用速度与性价比切换）；
- 存储服务：云服务器本地存储（数据库备份、日志文件）。

7.3 部署流程

1. 前后端代码分别打包为 Docker 镜像，前端采用 Nginx 镜像打包，后端采用 Python 3.10 镜像打包；
2. 编写 Docker Compose 配置文件，包含前端、后端、MySQL、Redis、Nginx 服务，配置端口映射、数据卷、网络；
3. 云服务器安装 Docker 与 Docker Compose，拉取镜像并启动容器；
4. 配置 Nginx 反向代理，将域名/公网 IP 映射到前端与后端服务；
5. 开启 MySQL 自动备份，配置后端日志分割，完成部署。